SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

ARYAN PATIL 25070122051 CSEA2

Subject: Programming with Java

ASSIGNMENT No : 10

TITLE : Write a Java program to demonstrate the use of abstract classes and abstract methods

Problem Statement

Write a Java program to demonstrate the use of abstract classes and abstract methods

Learning Objectives

To implement abstraction using abstract classes and abstract methods.

Theory

Abstract classes provide partial implementation of a program and cannot be instantiated directly. They may contain both abstract and concrete methods. Abstract methods declare functionality that must be implemented by subclasses. Abstraction hides implementation details while exposing only essential functionality. This improves code flexibility and maintainability in object-oriented programming.

Outcome

Exercise

1. Create an abstract **Payment class** and implement Credit Card and UPI payment classes.?

```java
abstract class Payment {
}

class CreditCardPayment extends Payment {

    void makePayment(double amount) {
        System.out.println("Paid Rs." + amount + " using Credit Card");
    }
}

class UPIPayment extends Payment {

    void makePayment(double amount) {
        System.out.println("Paid Rs." + amount + " using UPI");
    }
}

public class PaymentDemo {
    Run main | Debug main
    public static void main(String[] args) {

        Payment p1 = new CreditCardPayment();
        Payment p2 = new UPIPayment();

        p1.makePayment(5000);
        p2.makePayment(1500);
    }
}
```

OUTPUT-

```
PROBLEMS 14   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
Paid Rs.5000.0 using Credit Card
Paid Rs.1500.0 using UPI
```

2. Create an abstract **FoodOrder** class with an abstract method calculateBill(). Implement two subclasses, **DineInOrder** and **TakeAwayOrder**, to calculate and display the total bill based on the order type.

```java
abstract class FoodOrder {

    abstract void calculateBill();
}

class DineInOrder extends FoodOrder {

    double amount = 500;

    void calculateBill() {
        System.out.println("Dine-In Bill: Rs." + amount);
    }
}

class TakeAwayOrder extends FoodOrder {

    double amount = 500;
    double packingCharge = 20;

    void calculateBill() {
        System.out.println("Takeaway Bill: Rs." + (amount + packingCharge));
    }
}

public class FoodOrderDemo {
    public static void main(String[] args) {

        FoodOrder f1 = new DineInOrder();
        FoodOrder f2 = new TakeAwayOrder();

        f1.calculateBill();
        f2.calculateBill();
    }
}
```

OUTPUT-

```
PROBLEMS 14   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
Dine-In Bill: Rs.500.0
Takeaway Bill: Rs.520.0
```

Conclusion

The program was successfully implemented to demonstrate the use of **abstract classes** and **abstract methods** in Java. It showed how abstraction hides implementation details while providing a common structure for subclasses. The practical helped in understanding how different classes can implement the same abstract methods in their own way, improving code flexibility, reusability, and maintainability in object-oriented programming.